

Z3 Validator Component

Langflow custom component (RequirementsValidatorComponent) — consistency verification using AST constraints generated by the formalization component.

```
import json
import z3

from langflow.custom import Component
from langflow.io import MessageTextInput, Output
from langflow.schema import Message, Data

class RequirementsValidatorComponent(Component):

    display_name = "Z3 Validator"
    description = (
        "Consistency verification using AST constraints "
        "generated by the formalization component."
    )

    inputs = [
        MessageTextInput(
            name="input_data",
            display_name="Formal JSON",
            info="JSON generated by the formalization prompt"
        )
    ]

    outputs = [
        Output(
            name="output",
            display_name="Validation Result",
            method="run_validation"
        )
    ]

    def clean(self, value):

        if isinstance(value, list) and len(value) > 0:
            value = value[0]

        if isinstance(value, Message):
            value = value.text

        elif isinstance(value, Data):
            payload = value.data

            if isinstance(payload, dict):
                value = (
                    payload.get("text")
                    or payload.get("result")
                    or payload
                )
            else:
                value = payload
```

```

    if isinstance(value, dict):
        value = json.dumps(value)

    if value is None:
        return ""

    text = str(value).strip()

    if text.startswith("`json`"):
        text = text[7:]
    elif text.startswith("`"):
        text = text[3:]

    if text.endswith("`"):
        text = text[:-3]

    return text.strip()

def get_symbol(self, name, declared_symbols):

    if name not in declared_symbols:
        raise Exception(
            f"Undeclared symbol detected: {name}"
        )

    return declared_symbols[name]

def ast_to_z3(self, node, declared_symbols):

    node_type = node["type"]

    if node_type == "VAR":
        return self.get_symbol(
            node["name"],
            declared_symbols
        )

    if node_type == "NOT":
        return z3.Not(
            self.ast_to_z3(
                node["operand"],
                declared_symbols
            )
        )

    if node_type == "AND":

        operands = []

        for operand in node["operands"]:
            operands.append(
                self.ast_to_z3(
                    operand,
                    declared_symbols
                )
            )

```

```

        return z3.And(*operands)

    if node_type == "OR":

        operands = []

        for operand in node["operands"]:
            operands.append(
                self.ast_to_z3(
                    operand,
                    declared_symbols
                )
            )

        return z3.Or(*operands)

    if node_type == "IMPLIES":

        return z3.Implies(
            self.ast_to_z3(
                node["antecedent"],
                declared_symbols
            ),
            self.ast_to_z3(
                node["consequent"],
                declared_symbols
            )
        )

    if node_type == "IFF":

        return (
            self.ast_to_z3(
                node["left"],
                declared_symbols
            )
            ==
            self.ast_to_z3(
                node["right"],
                declared_symbols
            )
        )

    raise Exception(
        f"Unsupported AST node type: {node_type}"
    )

def build_signature(self, requirement):

    declared_symbols = {}

    signature = requirement.get(
        "z3_signature",
        {}
    )

```

```

    predicates = signature.get(
        "predicates",
        []
    )

    constants = signature.get(
        "constants",
        []
    )

    for predicate in predicates:

        name = predicate.get("name")

        if not name:
            continue

        arity = predicate.get("arity", 0)

        if arity == 0:
            declared_symbols[name] = z3.Bool(name)

    for constant in constants:

        if not isinstance(constant, dict):
            continue

        name = constant.get("name")

        if name:
            declared_symbols[name] = z3.Bool(name)

    return declared_symbols

def run_validation(self) -> Message:

    try:

        text = self.clean(self.input_data)
        data = json.loads(text)

        requirements = data.get(
            "requirements",
            []
        )

        if not requirements:

            return Message(
                text=json.dumps(
                    {
                        "status": "UNKNOWN",
                        "consistent": False,
                        "message": "No requirements found."
                    },
                    indent=4
                )
            )

```

```

    )

    solver = z3.Solver()

    tracked_requirements = {}

    requirement_ids = []
    traceability = []
    invalid_constraints = []

    total_constraints = 0

    for req in requirements:

        requirement_id = (
            req.get("requirement_id")
            or req.get("id")
            or "UNKNOWN"
        )

        requirement_ids.append(
            requirement_id
        )

        declared_symbols = (
            self.build_signature(req)
        )

        ast_constraints = req.get(
            "ast_constraints",
            []
        )

        for ast in ast_constraints:

            try:

                expr = self.ast_to_z3(
                    ast,
                    declared_symbols
                )

                track_symbol = z3.Bool(
                    f"track_{requirement_id}"
                )

                tracked_requirements[
                    str(track_symbol)
                ] = requirement_id

                solver.assert_and_track(
                    expr,
                    track_symbol
                )

                traceability.append(
                    {

```

```

        "requirement_id":
            requirement_id,
        "ast":
            ast
    }
)

total_constraints += 1

except Exception as ex:

    invalid_constraints.append(
        {
            "requirement_id":
                requirement_id,
            "ast":
                ast,
            "error":
                str(ex)
        }
    )

if total_constraints == 0:

    return Message(
        text=json.dumps(
            {
                "status": "UNKNOWN",
                "consistent": False,
                "requirements":
                    requirement_ids,
                "constraints": 0,
                "message":
                    "No AST constraints found.",
                "traceability":
                    traceability,
                "invalid_constraints":
                    invalid_constraints
            },
            indent=4
        )
    )

result = solver.check()
unsat_core = []

if result == z3.unsat:

    unsat_core = [
        tracked_requirements[
            str(x)
        ]
        for x in solver.unsat_core()
        if str(x) in tracked_requirements
    ]

if result == z3.sat:

```

```

        output = {
            "status": "SAT",
            "consistent": True,
            "requirements": requirement_ids,
            "constraints": total_constraints,
            "message": "No logical contradiction detected.",
            "traceability": traceability,
            "invalid_constraints": invalid_constraints
        }

    elif result == z3.unsat:

        output = {
            "status": "UNSAT",
            "consistent": False,
            "requirements": requirement_ids,
            "constraints": total_constraints,
            "message": "Logical contradiction detected.",
            "unsat_core": unsat_core,
            "traceability": traceability,
            "invalid_constraints": invalid_constraints
        }

    else:

        output = {
            "status": "UNKNOWN",
            "consistent": False,
            "requirements": requirement_ids,
            "constraints": total_constraints,
            "message":
                "Logical result inconclusive.",
            "traceability":
                traceability,
            "invalid_constraints":
                invalid_constraints
        }

    return Message(
        text=json.dumps(
            output,
            indent=4
        )
    )

except Exception as ex:

    return Message(
        text=json.dumps(
            {
                "status": "ERROR",
                "consistent": False,
                "message": str(ex)
            },
            indent=4
        )
    )

```

```
)  
  
if result == z3.unsat:  
    unsat_core = [  
        tracked_requirements[str(x)]  
        for x in solver.unsat_core()  
        if str(x) in tracked_requirements  
    ]  
  
    print("UNSAT CORE:", unsat_core)
```